

课程笔记

LightRAG 知识图谱双层检索与 LangFuse

基于公开课程资料整理

2025

LightRAG 知识图谱 LangFuse API 监控

视频作者/频道：五道口纳什

发布日期：2025

视频时长：约 19 分钟

视频链接：<https://www.bilibili.com/video/BV1TkmqBKEid>

项目主页：<https://github.com/hqh1025/ai-course-notes>

目录

1	如何快速上手一个 Agent 项目	2
1.1	方法论总结	2
1.2	本章小结	2
2	LightRAG 原理	2
2.1	概述	2
2.2	Indexing 流程：构建知识图谱	2
2.3	Retrieval 流程：双层检索	3
2.4	本章小结	3
3	LangFuse：本地部署的 API 监控工具	3
3.1	LangFuse 简介	3
3.2	部署与配置	3
3.3	通过 Traces 理解 Workflow	4
3.4	本章小结	4
4	LightRAG 核心 Prompt 解析	4
4.1	Entity Extraction System Prompt	4
4.2	Continue Extraction User Prompt	4
4.3	Keywords Extraction Prompt	4
4.4	RAG Response Prompt	4
4.5	本章小结	5
5	LightRAG 本地存储结构	5
6	总结与延伸	5

1 如何快速上手一个 Agent 项目

1.1 方法论总结

作者总结了快速理解和上手一个 Agent 项目的三步路径：

1. **理解基本概念**：读论文或项目框架介绍，理解 Workflow / Pipeline 设计
2. **理解 Prompt**：Agent 项目中大量环节都由 LLM 扮演——理解了 Prompt 就理解了各环节的输入、输出和处理逻辑
3. **使用监控工具**：通过 LangFuse 等工具 trace 所有 API 调用，可视化理解整个流程

Prompt 是理解 Agent 项目的钥匙

对于一个 Agent 的 Workflow，很多子环节都是 LLM 来扮演的。将每个环节理解为 $F(X; \text{Prompt})$ —— F 是 LLM, X 是 Query, Prompt 是 System/User Prompt。理解了 Prompt，就理解了各环节的处理逻辑。

1.2 本章小结

概念 → Prompt → 监控工具，这是一条高效的 Agent 项目理解路径。

2 LightRAG 原理

2.1 概述

LightRAG 是一篇 EMNLP 2025 的学术论文，也是 Paper2Slides 作者的另一个项目。它是一个基于知识图谱的 RAG 系统，相比传统向量 RAG 具有更强的语义理解能力。

LightRAG 的核心特性

- 基于 **Knowledge Graph** 的索引（而非纯向量索引）
- **双层检索**：Low-Level（局部，特定实体）+ High-Level（全局，主题和概念）
- **增量更新**：新文档无需重建整个 Graph，只需执行提取步骤并合并

2.2 Indexing 流程：构建知识图谱

对每个文档执行以下步骤：

1. **Chunking**：将文档分块
2. **Entity & Relationship Extraction**：基于 LLM 从每个 Chunk 提取实体和关系
3. **Profiling**：基于 LLM 对实体和关系生成文本描述
4. **Deduplication**：对整个知识图谱做去重合并
5. **Embedding**：对所有实体和关系的 Key 构建 Embedding Vector

知识图谱中的数据结构：

- **Entity**：名字、类型、描述（Profiling 生成）、原始 Chunk ID
- **Edge**：Source、Target、关键词、描述、原始 Chunk ID

LightRAG vs. GraphRAG

LightRAG 相比 GraphRAG 的一个重要优势是**增量更新**——新文档只需执行相同的提取步骤，合并实体和边即可。而 GraphRAG 需要重新进行社区发现，代价高昂。

2.3 Retrieval 流程：双层检索

1. 基于 Query，LLM 生成 **Low-Level Keywords**（特定实体）和 **High-Level Keywords**（主题/概念）
2. 基于 Embedding 相似度在知识图谱中匹配相关节点
3. 通过一**跳**邻居拓展 Subgraph
4. 获取三种类型的 Context：实体、关系、原始 Chunk
5. Context + Query → LLM 生成最终回答

2.4 本章小结

LightRAG 通过知识图谱索引 + 双层检索实现了比纯向量 RAG 更强的语义理解能力，且支持增量更新。

3 LangFuse：本地部署的 API 监控工具

3.1 LangFuse 简介

LangFuse 是一个可本地部署的 LLM API 监控工具，用于 trace 所有的模型调用过程。

3.2 部署与配置

1. 下载 Docker Compose 文件并启动服务
2. 本地访问 localhost:3000
3. 创建组织和项目
4. 获取 Public Key、Secret Key 和 Base URL
5. 在 .env 中配置以上信息

LightRAG **原生集成了** LangFuse——只要检测到环境变量，就会自动将 OpenAI Client 替换为 LangFuse Client，在后台记录完整的 traces。

3.3 通过 Traces 理解 Workflow

在 LangFuse 的 Traces 面板中，可以看到 LightRAG 的所有 API 调用：

Insert (Indexing) 阶段：

1. 对文档创建 Embedding
2. 第一次 Generation：提取实体和关系 (Entity Extraction System/User Prompt)
3. 第二次 Generation：Self Correction (继续提取遗漏的或格式不正确的内容)
4. 大量 Embedding 调用：对节点和边创建 Embedding

Query (Retrieval) 阶段：

1. Generation：提取 High-Level 和 Low-Level 关键词 (Keywords Extraction Prompt)
2. Embedding Search：基于关键词匹配实体和关系
3. 图遍历：找到相关实体的邻居节点
4. Generation：基于 Context + Query 生成最终回答 (RAG Response Prompt)

Response Cache

LightRAG 默认启用 Response Cache——如果重复执行相同的 Query，不会再次调用 API。这在开发调试时需要注意。

3.4 本章小结

LangFuse 提供了强大的可视化 trace 能力，是理解 Agent 项目内部工作机制的利器。

4 LightRAG 核心 Prompt 解析

4.1 Entity Extraction System Prompt

负责从 Chunk 中提取实体和关系，包含 Few-Shot 示例。

4.2 Continue Extraction User Prompt

基于第一次提取的结果，做 Self Correction——识别和提取遗漏的或格式不正确的内容。

4.3 Keywords Extraction Prompt

负责从 Query 中提取 High-Level 和 Low-Level 关键词，用于后续的向量搜索。

4.4 RAG Response Prompt

基于检索到的 Context (实体、关系、Chunk) 和用户 Query，生成最终回答。

4.5 本章小结

LightRAG 的核心 Prompt 包括：实体关系提取、Self Correction、关键词提取、RAG 回答生成——每一个都对应 Workflow 中的一个关键环节。

5 LightRAG 本地存储结构

运行 LightRAG 后，在工作目录下会生成以下文件：

- `graph.graphml`: XML 格式的知识图谱（实体 + 关系）
- `kv_store_*`: 键值存储（Chunk、Entity、Relation 的 K-V 对）
- `vdb_*`: 向量数据库（默认使用 Nano Vector DB）
 - `vdb_chunks`: Chunk 的 Embedding
 - `vdb_entities`: Entity 的 Embedding
 - `vdb_relationships`: Relation 的 Embedding

检索时的数据流：关键词 → Embedding Search (VDB) → 图遍历 (GraphML) → 补充原始文档 (KV Store) → 组装 Context → LLM 生成回答。

6 总结与延伸

1. **快速上手 Agent 项目的路径**：概念 → Prompt → 监控工具 (LangFuse)。
2. **LightRAG 是一个基于知识图谱的 RAG 方案**，相比纯向量 RAG 具有更强的语义理解和增量更新能力。
3. **双层检索** (Low-Level + High-Level) 复用了 GraphRAG 中 Local 和 Global 检索的思想，但实现更轻量。
4. **LangFuse 是理解 Agent 内部机制的利器**：通过 trace 所有 API 调用，可以清晰看到每个环节的输入输出。
5. **核心 API**：`ainsert` (增量索引) 和 `aquery` (检索回答) ——对应 Indexing 和 Retrieval 两大流程。